
SECTION: _index.md

Introduction

The GitLab team handbook is the central repository for how we run the company. Printed, it consists of over 2,000 pages of text. As part of our value of being transparent the handbook is open to the world, and we welcome feedback. Please make a merge request to suggest improvements or add clarifications. Please use issues to ask questions.

For a very specific set of internal information we also maintain an Internal Handbook

Handbook Contents

{{< handbook-data-toc >}}

{{% include "includes/take-gitlab-for-a-spin.md" %}}

SECTION: about/_index.md

History of the handbook

The handbook started when GitLab was a company of just ten people to make sharing information efficient and easy.

We knew that future GitLab team-members wouldn't be able to see emails about process changes that were being sent before they joined and that most of the people who would eventually join GitLab likely hadn't even heard of us yet.

The handbook was our way of ensuring that all of our company information was accessible to everyone regardless of when they became part of the team.

Advantages

At GitLab our handbook is extensive and keeping it relevant is an important part of everyone's job.

It is a vital part of who we are and how we communicate.

We established these processes because we saw these benefits:

1. Reading is much faster than listening.
1. Reading is async, you don't have to interrupt someone or wait for them to become available.
1. Talent Acquisition is easier if people can see what we stand for and how we operate.
1. Retention is better if people know what they are getting into before they join.
1. On-boarding is easier if you can find all relevant information spelled out.

1. Teamwork is easier if you can read how other parts of the company work.
1. Discussing changes is easier if you can read what the current process is.
1. Communicating change is easier if you can just point to the diff.
1. Everyone can contribute to it by proposing a change via a merge request.

One common concern newcomers to the handbook express is that the strict documentation makes the company more rigid.

In fact, writing down our current process in the handbook has the effect of empowering contributors to propose change.

As a result, this handbook is far from rigid. You only need to look at the handbook commits list to see the evidence. Every attempt is made to document guidelines and processes in the handbook. However, it is not possible to document every possible situation or scenario that could potentially occur. Just because something is not yet in the handbook does not mean that it is allowed. GitLab will review each team member's concern or situation based on local laws to determine the best outcome and then update the handbook accordingly. If you have questions, please discuss with your manager or contact the People Success team.

Handbook Interpretation

The handbook is subject to interpretation. We do our best to be as clear as possible to minimize confusion and/or misinterpretation. We also recognize that we have a global audience and that may bring different interpretations. If you have any questions or need further clarification please check with the content owner of the page. When in doubt please reach out and ask.

Remember that everything is in draft at GitLab and subject to change, this includes our handbook.

Count handbook pages

It's easy to see that the handbook is large, but have you ever wondered just how large? The handbook is over two thousand pages long. That's a lot of good info!

Historical Word and Page Counts

about.gitlab.com/handbook

```
{{< handbook-counts site="about" >}}
```

handbook.gitlab.com/handbook

```
{{< handbook-counts site="handbook" >}}
```

Methodology

Word and page counts are determined through a simple two-step process:

1. Count the number of words in the handbook. This can be done by running `find sites/handbook/source/handbook -type f -name "*.md" -o -name "*.md.erb" | xargs wc -w` from the root of the repository.

1. Count the number of pages in the handbook. This can be done by running `grep -l -r "\n" | wc -l` from the root of the repository.

Note: If you need to go back to an earlier version of the handbook, use `git checkout git rev-list -n 1 --first-parent --before="2021-07-02 00:00" master` specifying the next day after the day you want.

[View Statistics](#)

On this page you can see handbook trends and discover popular pages that you may not know about.

GitLab uses Snowplow to track handbook usage; the information can be viewed on Tableau.

[More about the handbook](#)

We've gathered some information about the handbook here, but there's still more elsewhere.

- [Handbook usage](#)
- [Handbook editing examples](#)

SECTION: about/contributing.md

{{% alert title="Note" color="primary" %}}

Please see the repository contributing file for information on the Developer Certificate of Origin + License when contributing to GitLab.

{{% /alert %}}

You can help to improve the GitLab Handbook website by submitting merge requests to the handbook project.

Thank you for your interest and help!

We don't mind people inside and outside the company making suggestions to change significant things.

Feel free to make a proposal, we can discuss anything. If we don't agree, we'll feel free not to merge it, and we'll thank you for caring about it.

We want to create a welcoming environment for everyone who is interested in contributing. Please visit our [Code of Conduct](#) page to learn more about our commitment to an open and welcoming environment.

SECTION: about/direction.md

[Overview](#)

The GitLab Handbook is the single source of truth for how we operate at GitLab, including

processes, policies, and product direction. In keeping with our value of transparency, the GitLab Handbook is entirely open to the world. We welcome feedback from the community and hope that it serves as inspiration for other current or future companies. The GitLab Handbook is also an incredible talent acquisition tool, providing candidates with valuable insight into how GitLab runs as a company.

A sub-section of the handbook.gitlab.com website, the GitLab Handbook specifically refers to content that is in the `/handbook/` namespace of the website. The overall user experience and architecture of the GitLab Handbook is a shared responsibility. Feedback and questions can be directed to the handbook Slack channel.

Target Audience

GitLab Team Members: Every GitLab team member is responsible for using and updating our handbook. It is the central repository for process documentation and product direction.

Leadership: GitLab leadership uses the handbook like any other member of the team, but additionally needs to reference the content during presentations to stakeholders or investors. Since everything we do is open to the public, members of leadership teams outside of GitLab may also use the GitLab Handbook as reference or inspiration for their own team processes and policies.

Potential Applicants: Candidates for job opportunities at GitLab use the handbook to learn more about expectations for the role, compensation and benefits, GitLab's company values, and other policies. GitLab team members also use the handbook extensively to share specific, relevant information with potential applicants, making it a powerful talent acquisition tool in itself.

Current and Potential Users: GitLab's product direction, strategy, and vision are documented in the handbook alongside our product and engineering processes. This allows current users a glimpse into GitLab's future priorities and can help potential users make an informed decision related to adopting GitLab as a tool.

Where we are Headed

At GitLab, we encourage everyone to work handbook first in order to promote asynchronous collaboration and documentation. Working this way has its challenges, not the least of which is the time and effort involved in making a change. While this extra investment can encourage contributors to be more considered and deliberate with their changes, at a certain point it discourages meaningful collaboration and works against our goals.

Our hope is that the GitLab Handbook is something that others want to emulate. To facilitate that, we want to ensure that any user can easily use and update the handbook. Ideally, the handbook has:

- Organized, and up-to-date content
- Fast, predictable deployments
- A clean, scalable information architecture and modern codebase

What's Next & Why

We are focused on maintaining the handbook's fast (pipeline under 10 minutes) and stable (minimal pipeline failures) state.

Anything further is under consideration.

As there is no full time DRI, the following are on hold:

- Improved reading and sharing experience: The content needs to remain the primary focus but it should be easier to search, navigate, and share the content.
- Freshness tools: To ensure that content is relevant, up-to-date, not duplicative, and to give readers the tools they need to flag content that doesn't meet those criteria.
- Other Issues: Handbook issues

What is Not Planned Right Now

We are not currently investigating a transition to a separate, external content management system or publishing platform.

The needs of the GitLab Handbook have outgrown what can be handled in a wiki (or similar) product, so we are not planning to migrate any content into that format.

Since the content is changing quite literally every day, we are not looking to generate a digital or printed book from the GitLab Handbook content.

The GitLab Handbook is not currently optimized for serving as a searchable Knowledge Base or FAQ repository similar to what you would find on Stack Overflow.

SECTION: about/escalation.md

For information on team members' roles and responsibilities, see [Content Websites](#) page.

Introduction

The Handbook is a critical part of empowering team members to do their jobs effectively. As such, we have a group of team members who assist in resolving issues affecting all team members.

Reporting an issue

If you're looking for general help, please see the [editing handbook](#) page.

Any work stopping issues should be reported in the handbook-escalation channel in Slack. Otherwise, consider creating an issue in the relevant content sites repository and posting in the handbook Slack channel.

When to escalate an issue

Issues should only be escalated if it relates to:

1. Default branch being broken
1. Broken infrastructure
1. Time sensitive updates to the handbook where there is an issue in making the update

"Keep main green" group

If after posting in handbook-escalation, you are not receiving a response, please ping the issue triager if it's within their working hours.

If they are not available, the following team members (in the table below) have volunteered to assist when available. Please take into consideration the listed timezone where they are generally located and their Slack status on whether they're out of office.

Additionally, any GitLab team member can volunteer to join the handbook-escalation channel and help out.

Slack username	Usual timezone(s)
@Arty-chan	Pacific Time (UTC-8/-7)
@mayra	Central Time (UTC-6)
@David Nelson	Central Time (UTC-6/-5)
@A.J. Romaniello	Eastern Time (UTC-5/-4)
@Max Woolf	GMT (UTC+0/+1)
@Filip	Central European (UTC+1/+2)
@Marcin SJ	Central European (UTC+1/+2)
@arpit	India Time (UTC+5:30)
@jaime	Australia Eastern Time (UTC+10/+11)

Expectations for the group

1. Make sure you are in and do not mute the handbook-escalation channel.
1. When an issue is reported:
 1. Acknowledge the team member and let them know you are looking into it.
 1. You can check on production, incidents-dotcom, and is-this-known to see if it's a known issue with infrastructure or other problems.
 1. Provide an update as soon as you are able to confirm their problem.
 1. You can also post updates in mr-buddies and/or handbook as appropriate.
 1. Offer to have a Zoom call to help replicate or resolve the issue if it is not straight forward.
 1. Resolve the problem, or provide feedback to the team member on how they can resolve it.
 1. If you do not believe you can resolve it and need further assistance, consider any or all of the following:
 1. Ping another member of the Keep main green group.
 1. Ping the handbook DRI, or a member of the code maintainer group.
 1. Create a bug issue with details.

When to hand over to Reliability Engineering

Handbook escalation deals specifically with matters relating to the content-sites projects. If a reported issue relates to the GitLab product or the infrastructure running the handbook,

then it should be escalated to the Reliability Engineering team.

To report an incident follow the instructions on the Incident Management page:

<<https://handbook.gitlab.com/handbook/engineering/infrastructure/incident-management/reporting-an-incident>>

Common Incidents and Tips

Failing pipeline

Please see the handbook editing page for guidance and examples.

Managing broken main alerts in handbook-escalation

All broken CI pipelines for the main branch of the handbook repository are automatically posted in the Slack channel.

These reports should be investigated and addressed where needed.

Once a report has been looked at, please leave a comment stating the nature of the problem, action taken, and add a · reaction to the message to show that it has been handled.

If for some reason there is a large amount of failures resulting in spamming the channel, the error reporting can be turned off in the repository settings: <<https://gitlab.com/gitlab-com/content-sites/handbook/-/settings/integrations>>

Stuck Merge Train

To see the status of the merge train (useful when team members are reporting that their MRs seem 'stuck' on the train), see this issue to check the status and perform a workaround, if necessary.

TL;DR for workaround: If the first/oldest MR iid in the FIFO list (sort=asc by ID) is actively running a pipeline and eventually gets merged, then things are moving along, just slowly. If the first one in the list isn't currently running any pipeline, remove it from the train and re-add it (it should go to the end).

Runbook for about.gitlab.com

While the handbook is no longer on about.gitlab.com, some of the information in the runbook for about.gitlab.com incident handling may be helpful.

SECTION: about/handbook-usage.md

Flow structure

1. A (process) problem comes up, frequently in an issue or chat.
1. A proposal is made in a merge request to the handbook.
1. Once merged, the change is announced by linking to the diff in the MR or commit. Major ones are posted in the whats-happening-at-gitlab slack channel. Medium ones are posted in the

handbook channel for visibility, with a one line summary of the change. If there was an issue, close it out with a link to the diff.

Sometimes you want to have real time editing of a proposal during a meeting and you need to use a Google Doc for that. When doing so the first item should be the URL of the handbook page this content will be moved to when the meeting is over.

Why handbook first

Documenting in the handbook before taking an action may require more time initially because you have to think about where to make the change, integrate it with the existing content, and then possibly add to or refactor the handbook to have a proper foundation. But, it saves time in the long run, and this communication is essential to our ability to continue scaling and adapting our organization.

This process is not unlike writing tests for your software. Only communicate a (proposed) change through a change to the handbook; don't use a presentation, email, chat message, or another medium to communicate the components of the change. These other forms of communication might be more convenient for the presenter, but they make it harder for the audience to understand the context and the implications for other potentially affected processes.

Having a "handbook first" mentality ensures there is no duplication; the handbook is always up to date, and others are better able to contribute.

Beyond being "handbook first," we are also "public handbook first." When information is internal-only, it should be captured in the internal handbook, but we default to the public handbook for anything that can be made public. This ensures that everyone has access to any information that can be SAFELY shared. This supports the GitLab values, including transparency, efficiency, and results. It also protects against the internal handbook becoming a home for information that should otherwise be public or a conflicting or duplicative source of truth.

<!-- blank line -->

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/EuGsen3FxC?start=991" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
```

<!-- blank line -->

When asked during an INSEAD case study interview (shown above) about challenges related to being all-remote, GitLab co-founder Sid Sijbrandij provided the following reply.

- > The biggest problem is GitLab not working handbook first. We have an amazing handbook that allows us to collaborate, onboard new people, and think collectively.
- >
- > However, it is counterintuitive to communicate changes to the handbook. The default of people, when they wish to communicate a change, is to send a Slack message, send an email, give a presentation, or tell people in a meeting · anything but make a change in the handbook.
- >
- > It's slower for them. It's quicker to use any other form. If they make a change in the handbook, they first have to find the relevant part of the handbook, they sometimes have to

adjust the handbook to make sure their change will fit in, they have to go through a technical process and a review or two, and they have to wait a bit before it's deployed.

>

> It's slower than any other option. However, it allows people that commit a change after that to build upon a change. When they take that extra time, it's building a foundation for the next thing.

>

> I think of it as brick laying. Every piece of information is a brick. At GitLab, there is a well-structured house, and everyone adds to that one house. Because we're pretty particular on how we build it, it has a strong foundation and we can build it very high.

>

> In every other company, they send the brick into the hands of people. Everyone is receiving bricks daily that they have to add to the house they're building internally. They forget things and things are unclear. A lot of context has to be created because there is no context around where to place the bricks.

>

> So, you can end up with a thousand houses that look quite different, that are all hanging a bit, and each time you add a brick to the top one pops out at the bottom. · GitLab co-founder Sid Sijbrandij

Scope of this handbook

The handbook is focused on any content that GitLab team members need to do their job. For other content that might be useful, link to the other source.

- All documentation that also applies to code contributions from the wider community should be in the GitLab project (for example in Contributing or the code review guidelines), not the Handbook, which is only for team members.
- For user facing company information, there's the main marketing website, and the blog.
- The handbook is for things concerning current and future GitLab team-members only. If something concerns users of GitLab, it should be documented in the GitLab documentation, the GitLab Development Kit (GDK), the CONTRIBUTING file or the PROCESS file.

Handbook guidelines

Please follow these guidelines and remind others of them.

How we use the guide every day

1. Most guidelines in this handbook are meant to help, and unless otherwise stated, are meant to help more than being absolute rules. Don't be afraid to do something because you don't know the entire handbook, nobody does. Be gentle when reminding people about these guidelines. For example say, "It is not a problem, but next time please consider the following guideline from the handbook."

1. If you ask a question and someone answers with a link to the handbook, they do this because they want to help by providing more information. They may also be proud that we have the answer documented. It doesn't mean that you should have read the entire handbook, nobody knows the entire handbook.

1. If you need to ask a team member for help, please realize that there is a good chance the majority of the community also doesn't know the answer. Be sure to document the answer to

radiate this information to the whole community. After the question is answered, discuss where it should be documented and who will do it. You can remind other people of this request by asking "Who will document this?"

1. When you discuss something in chat always try to link to a URL where relevant. For example, the documentation you have a question about or the page that answered your question. You can remind other people of this by asking "Can you please link?"

1. Remember, the handbook is not what we hope to do, what we should formally do, or what we did months ago. It is what we do right now. Make sure you change the handbook in order to truly change a process. To propose a change to a process, make a merge request to change the handbook. Don't use another channel to propose a handbook change (email, Google Doc, etc.).

1. The handbook is the process. Any sections with names like 'process', 'policies', 'best practices', or 'standard operating procedures' are an indication of a deeper problem. This may indicate a duplication between a prose description of a process and a numbered list description of the same process that should be combined in one description of the process.

1. When communicating something always include a link to the relevant (and up-to-date) part of the handbook instead of including the text in the email/chat/etc. You can remind other people of this by asking "Can you please link to the relevant part of the handbook?"

1. Everyone can join the handbook channel to stay up-to-date with changes to the handbook.

Screenshot the handbook instead of creating a presentation

Presentations are great for ephemeral content like board presentations. Evergreen content like a leadership training should be based on the handbook. This is an important element of working handbook-first.

In the creation of presentations for evergreen content, please screenshot the handbook and provide links to displayed pages rather than copy and pasting content (or formatting a slide specifically to mirror handbook information). This approach shows a bias towards asynchronous communication, and rationale for this is below.

1. It saves you the effort of needing to both update the handbook and create content for a presentation; the handbook is checked and improved as part of the preparation instead of extra work

1. The handbook will stay up to date so people don't look at an outdated information in a presentation

1. Seeing screenshots will confirm to people the content is in the handbook and it is up to date there

1. People get used to the structure of the handbook and can more easily find the relevant handbook section later on

1. The content is open for everyone to contribute to when it is in the handbook

1. The content is integrated with the rest of our processes and shown in context

1. Many more people will consume the content on a webpage than a presentation because it is easier to search and link

1. You're helping other organizations and students around the work by giving them an example how we do it

1. Also see some of the advantages of using our handbook

The presentation will look less polished, but the advantages outweigh that concern.

If a synchronous presentation is required, default to sharing your screen and viewing live

handbook pages over a slide presentation.

Searching the Handbook

Every GitLab Handbook page has a search field near the top of the page for searching. See the [Searching GitLab like a pro](#) page for tips on searching the handbook faster and more efficiently.

How to change or define a process

1. To change a guideline or process, suggest an edit in the form of a merge request.

1. When working to get your change merged quickly, make sure you are asking the appropriate team members with merge rights. Not sure who is responsible? Consult (and add to) the CODEOWNERS file in the relevant project.

- For example, CMO (Chief Marketing Officer) Staff members can approve any merge request that is specific to their function, while the CMO approves any merge request that is cross-functional marketing. Chief of Staff to the CEO/CEO approves any merge request that has cross-company impact, such as GitLab's Values page.

1. After it is merged you can post this in the [whats-happening-at-gitlab](#) slack channel if applicable. You can remind other people of this by asking "Can you please send a merge request for the handbook?"

1. When substantially changing handbook layout, please leave a link to the specific page of the review app that is directly affected by this MR. Along with the link, include as much info as possible in the MR description. This will allow everyone to understand what is the purpose of the MR without looking at diffs.

1. Keeping up with changes to the Handbook can be difficult, please follow the commit subject guidelines with a particular focus on your merge request's title, to ensure someone can quickly understand the MR's content.

1. Communicate process changes by linking to the merged diff (a commit that shows the changes before and after). If you are communicating a change for the purpose of discussion and feedback, it is ok to link to an unmerged diff. Do not change the process first, and then view the documentation as a lower priority task. Planning to do the documentation later inevitably leads to duplicate work communicating the change and it leads to outdated documentation. You can remind other people of this by asking "Can you please update the handbook first?"

1. When feasible, introduce process changes iteratively. It is important that you contribute to the handbook by making small merge requests. This will help gain adoption among the process's intended audience. We want to avoid significant process changes that are unnecessarily large, top-down, and disruptive. These types of process changes can disempower DRIs and cause people to focus on process rather than results.

1. Like everything else, our processes are always in flux. Everything is always in draft, and the initial version should be in the handbook, too. If you are proposing a change to the handbook, whenever possible, skip the issue and submit a merge request. (Proposing a change in a merge request is preferred over an issue description). Mention the people that are affected by the change in the merge request. In many cases, merge requests are easier to collaborate on since you can see the proposed changes.

1. If something is a limited test to a group of users, add it to the handbook and note as such. Then remove the note once the test is over and every case should use the new process.

1. If someone inside or outside GitLab makes a good suggestion invite them to add it to the

handbook. Send the person the URL of the relevant page and section and offer to do it for them if they can't. Having them make and send the suggestion will make the change and will reflect their knowledge.

1. When you submit a merge request, make sure that it gets merged quickly. Making single, small changes quickly will ensure your branch doesn't fall far behind master, creating merge conflicts. Aim to make and merge your update on the same day. Mention people in the merge request or reach them in Slack. If you get a suggestion for a large improvement on top of the existing one consider doing that separately. Create an issue, get the existing MR merged, then create a new merge request.

1. If you have to move content have a merge request that moves it and does nothing else. If you want to clean it up, summarize it, or expand on it do that after the moving MR is merged. This is much easier to review.

1. Try to add the why of a handbook process, what is the business goal, what is the inspiration for this section. Adding the why makes processes easier to change in the future since you can evaluate if the why changed.

1. When adding or changing a process in the engineering workflow, consider scheduling a retro X months out. The retro is an opportunity to consider the effects of the new process and determine if it needs further iteration or can be removed.

Considerations when adding an additional process

While not a requirement, it can be useful to schedule an async retro MR/issue X months out after the introduction of a new process.

Consider identifying an improvement target and timeframe to determine what success is. You can use the retro MR/issue to transparently communicate expectations as well to provide a means to iterate or remove the new process.

The creation of the retro MR/issue is an opportunity to think through:

1. What do we hope this new process accomplishes?

1. Is there a metric we can use to measure the effect of the new process?

1. Are there outcomes of the new process that would prompt us to roll it back?

The retro itself is an opportunity to document:

1. How do the participants in the process think things are going?

1. What additional costs are associated with the new process?

1. Were there any unforeseen side effects of the new process?

1. Do the participants have suggestions for ways to mitigate any costs realized by the new process, or further improvements in general?

Information architecture

Single Source of Truth

Think about the information architecture to eliminate repetition and have a Single Source of Truth (SSoT). Instead of repeating content cross-link it (each text has a hyperlink to the other piece). If you copy content please remove it at the origin place and replace it with a link to the new content. Duplicate content leads to more work by having to update the content

in multiple places as well as the need to remember where all of the out of date content lives. When you have a single source of truth it's only stored in a single system.

Make sure to always cross-link items if there are related items (elsewhere in the handbook, in docs, or in issues).

System of Record

A system of record (SoR) is the authoritative data source for a given data element or piece of information. It's worth noting that while it is possible to have a system of record that is also a single source of truth, simply just being a system of record doesn't directly imply it is the single source of truth.

Organized by Function and Results

The handbook is organized by function and result to ensure every item in it has a location and owner to keep it up to date.

- It's essential that we adhere to this hierarchy and that we not maintain separate structures for company training materials (such as onboarding materials, how-tos, etc.), videos, or other documentation.

- Adhering to this hierarchy is sometimes counter-intuitive.

We've learned over the years that keeping content in context helps to ensure consistency when making future updates.

- At times, a change of perspective may be desired.

In those cases, link to relevant sections of the handbook but don't include the content itself.

See the onboarding template (internal) as an example.

Or for example a list of Key Performance Indicators that links to performance indicators but doesn't duplicate definitions.

Style guide

Read through these style guidance below and the Writing Style Guidelines before editing the handbook.

Handbook pages

Keep your handbook pages short and succinct. Eliminate fluff and get right to the point with the shortest possible wording.

Keep in mind that the biggest challenge cited by new employees is the vast amount of information to take in during onboarding.

Avoid unstructured content

- Avoid unstructured content based on formats like Learning Playbooks, FAQs, lists of links (such as quick links), resource pages, glossaries, courses, videos, tests, processes, standard operating procedure, training, or how-to's.

These are very hard to keep up-to-date and are not compatible with organization per function and result. For example: Call it Contract Negotiation Handbook instead of Contract Negotiation Playbook

- Instead, put the answer, link, definition, course, video, or test in the most relevant place.

Use descriptive headings so that people can easily search for content.

- That said, please mix formats where and when appropriate in the handbook, even within a single page.

Utilizing multiple formats can be valuable, and different people may prefer certain formats over others.

- Worry only about the organization per function and result, not about how the page will look if you embed varying types and formats of content.

- Note: A weakness of FAQs is that questions are often asked in biased or leading ways. When possible, state facts as facts.

Use headings liberally

If a page includes more than two headings (especially if it's larger than a single "screen"), add an automatically generated Table of Contents (ToC) by copying line 6 to 10 in this MR. Headings should have normal capitalization: don't use ALL CAPS or TitleCase.

After a heading, leave one blank line; this is not required in the standard, but it is our convention.

Use contributable diagrams

Preference contributable diagrams over uploading images or other less contributable diagrams. This makes it easier for other people to suggest changes and contribute. Diagram options in Markdown include Mermaid and PlantUML.

Handbook First Competency

In an all-remote, asynchronous organization, each team member should practice handbook first. For more information on what it means to be handbook first, please refer to the why handbook first section of this page.

Skills and behaviors of handbook first as a Team Member:

- Actively contributes to the handbook.
- Start with a merge request
- Provides links to information from the handbook when answering questions and if the information doesn't exist in the handbook, then the team member adds it.
- Understands which information is internal-only, but puts all public information in the public handbook.

Skills and behaviors of handbook first as a People Leader:

- Holds their team and others accountable for being handbook first.
- Demonstrates leadership in being public handbook first with all information that is not internal-only.
- Enables new team members and managers on how to leverage the handbook as a resource.
- Serves as a role model for what it means to be handbook first.

Management

It is each department and team member's responsibility to ensure the handbooks (public

handbook, and internal handbook) stay current. The content in the handbook should be accurate and follow the same format as outlined in the Guidelines. For questions on who to submit a merge request to, or assistance with the handbook, please reach out on the handbook Slack channel.

If you need permissions to directly commit changes to the handbook, please submit a New Access Request issue and follow the process for access approval. Request a 'Maintainer' role for the appropriate handbook.

Merge Rights Guidelines

You need developer access to the relevant handbook project to merge MRs. All team members should have developer access by default. This page contains some tips and guidelines that you should keep in mind when merging.

Merge with confidence

You should feel confident merging any changes that pass the pipeline without worrying that you will break the handbook. The tests in the pipeline are designed to catch any major problems. The handbook projects are configured so that changes cannot be merged unless the pipeline passes. In the event that code is merged that does break the handbook in some way, follow the instructions for reporting an issue to the Handbook team.

Do not use merge immediately

Do not use the merge immediately feature! Even if your MR is important and time-sensitive, using this feature will create a lot of pain for everyone else. This feature should only be used when critical public information needs to be sent live as quickly as possible and should be approved by PR or Legal. If you don't have PR or Legal approval, don't use this feature.

More context on the technical reasons behind this:

- We want to have a fast pipeline for the default branch, which minimizes the time needed for changes to be deployed and appear live on the production site.
- In order to achieve this, we do not run any tests or linters on the default branch, because these are long-running jobs which would block a fast deployment.
- Instead, we rely on the Merge Train to ensure that all Merge Request changes have successfully passed all necessary test/lint jobs before being allowed to merge.
- So, if you use "merge immediately", none of the test/lint jobs will be run, which will result in a broken default branch if problems were introduced.
- This means that everyone who creates new MRs off of the default branch after this point will experience confusing pipeline failures which are not their fault, and this will continue until this situation is discovered and a fix is merged to the default branch.

When to get approval

Getting pinged to approve every small change to your page can be annoying, but someone changing a policy or procedure in the handbook without proper approval can have strong negative consequences. Use your best judgement on when to ask for approvals.

Whenever reasonable, practice responsibility over rigidity. When you expect a page owner will appreciate your changes, go ahead and merge them without approval. Always ping the code owners with an @mention comment to inform them of the changes. They will be happy their page was made better and they didn't need to waste time reviewing and approving the change. In the event that something isn't an improvement, we practice clean up over sign off.

Whenever appropriate, such as publishing a previously internal-only document, get approval from the code owner using the approval feature before merging changes.

Who can approve

You can identify the owner of a handbook page by visiting the page on the web version of the handbook and viewing who is in the Maintainers section on the page being edited (on the right hand side of the page, which may be hidden on a smaller browser window).

For some pages, the Maintainers section is empty. This is because the page is maintained by a group rather than individuals. To view which groups or individuals maintain a page, see the CODEOWNERS file of the relevant repository, such as the handbook's CODEOWNERS.

The page's code owners will also be visible when viewing the approval section of the merge request.

The page's code owner is the DRI for the page and has the final say for what appears in the handbook. When in doubt, get the DRI's permission before changing their page. Don't worry if the DRI is a C-level person. You can still assign your MRs to them, even if you are an individual contributor. This is because we prefer to communicate directly.

Have a peer review your changes

Unless it's a small change like a typo, always have another team member review your changes before you merge them.

Broad Permissions

Being a maintainer gives you access to much more than just the ability to merge. You can see a full list of permissions in the docs. Keep in mind that you'll have access to a broad set of settings and configuration for the project. Don't adjust any settings or make any structural changes.

Granting others maintain access

Do not grant people maintainer access without an Access Request. As a maintainer you have the ability to grant others maintainer access. Don't do so without following the Access Request process to garner the appropriate approvals and create the necessary documentation.

Description Templates for Issues and Merge Requests

The description templates for issues and merge requests in the various projects are respectively located in the `.gitlab/issuetemplates` and `.gitlab/mergerequesttemplates` directories, and can be edited and changed with a new merge request.

Changes to the default templates in `Default.md` require review by the Office of the CEO or the CEO. See `.gitlab/CODEOWNERS` for the current list of approvers.

The Internal handbook

Where can it be found?

- Internal Handbook Project
- Live Website

Why is there a internal handbook?

As a company, we are public by default, but there are things that we cannot discuss publicly. The Internal Handbook is a space where team members can share internal information. Anything that is not considered internal only, should be in GitLab's public handbook. Anything that is limited access should not be added to the internal handbook as the internal handbook is accessible by all team members.

The process of using this handbook

Only add items to the internal handbook that fall into the not public category. Everything else should be added to our public company handbook.

Accessing the internal handbook group

All team members have access to the internal handbook through Okta as baseline access during onboarding. Login to your Okta dashboard and click on the GitLab Internal Handbook tile. You will have to authenticate with Okta first.

Questions using the internal handbook

- Questions about what should be in the internal handbook vs in the public handbook? See the safe framework or ask in safe.
- Volunteering to help with enhancing it? Join the internal-handbook channel.

KPI

The Engineering team and all sub-teams track Handbook Update Frequency as a KPI, with varying targets per team. Currently, Engineering is the only Division tracking Handbook update frequency, so as to analyze and observe the effectiveness of this KPI.

External use of the Handbook

Remember that, like virtually everything we do, our handbook is open source, and we expect that

other companies may use it as inspiration for their own documentation and practices. That said, the handbook should always be specific on what we do, not who we want to be, and every company will need to fill out their own handbooks over time. Our handbook falls under the Attribution-ShareAlike 4.0 International license.

Inspired by GitLab

If your company has been inspired by GitLab's handbook, we would love to know what inspired you. Please see our Inspired by GitLab page.

Make it worthwhile

Another company asked how we managed to work with the handbook because at their company it wasn't working: "There are many occasions where something is documented in the knowledge base, but people don't know about it because they never bothered to read or search. Some people have a strong aversion against what they perceive as a 'wall of text'."

We attempt to cover this in GitLab's guide to embracing a handbook-first approach to documentation.

To ensure that people's time is well spent looking at the handbook we should follow the 'handbook guidelines' above, and also:

1. Follow the writing style guide.
1. Have a great search function, plus make it public so you can Google search.
1. Test people on their knowledge during onboarding.
1. Give real examples.
1. Avoid corporate speak, describe things like you're talking to a friend. For more, see our communications guide on Simple Language.

Wiki handbooks don't scale

Company handbooks frequently start as wikis.

This format is more comfortable for people to work in than a static website with GitLab Merge Requests and GitLab Pages.

However wikis tend to go stale over time, where they are badly organized and get out of date.

In wikis it is impossible to make proposals that touch multiple parts of a page and/or multiple pages. Therefore it is hard to reorganize the handbook. Because GitLab Merge Requests and GitLab Pages are based on distributed version control you can split the role of submitter and approver. This allows for a division of work that keeps the handbook up to date:

- Anyone who can read the handbook can make a proposal
- Leaders (who tend to be short on time) only have to approve such a proposal

Wikis also do not encourage collaboration on changes, because there is no way to discuss a proposed change like there is with merge requests.

Some wikis make it hard to view and/or link to diffs of changes, which is needed to communicate decisions.

Sid and Paul Discussion on the benefits of markdown handbooks and limitations of wikis

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/eLc8PIDucw" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
```

SECTION: about/maintenance.md

Overview

This area has traditionally been referred to as "the handbook", but over time has grown in scope to include multiple sites, projects, repos, and types of content.

Therefore, we are using the term "content websites" here to avoid ambiguity and properly frame discussions around this scope of responsibility.

See our direction page for more.

If you need help, please see the editing handbook section, or escalation information if it's urgent.

Team Structure

The maintainer of this page (as indicated in the sidebar) is considered the DRI for GitLab's "content websites". At present, the roles and responsibilities are as follows:

Role	Owner	Current Individual(s)	Responsibilities
-----	-----	-----	-----
Handbook DRI	Learning & Development	Tre Ely, Jamie Allen	Prioritize work, make decisions on content sites, coordinate major changes
Keep pipeline green	Group of volunteers	See Escalation page	Help fix the pipeline if jobs are failing (as needed)
Code Maintainer ^[1]	Group of volunteers	DRI, plus [@gitlab-com/content-sites/handbook-tools]	Code reviews, escalation point for "Keep pipeline green" group, and as time allows, development work

[¹]: Note: Team members are added to the handbook-tools group as Maintainer or Owner. The group has Maintainer access to the handbook projects, and the Docsy theme project.

This page further documents the scope and responsibilities of the DRI and their engineering reports.

What are the content websites?

1. The public handbook.gitlab.com website:

1. While often referred to as "the handbook", this website also serves a wide variety of other content including the job families, and the TeamOps pages.

1. handbook.gitlab.com is primarily backed by the gitlab-com/content-sites/handbook project and repo.

1. Data (YML) files currently resides in the gitlab-com/www-gitlab-com repository.

1. The "Internal Handbook" at internal.gitlab.com:

1. This website contains content that falls into the not public category. More details are available in the Internal Handbook usage page

1. The Internal Handbook is backed by the gitlab-com/content-sites/internal-handbook project and repo.

1. The theme for the handbook sites is in the gitlab-com/content-sites/docsy-gitlab project.

What are NOT content websites?

1. The about.gitlab.com marketing website.

1. The gitlab.com site.

1. The docs.gitlab.com product documentation site.

1. Any other GitLab-managed or GitLab-owned website other than what is described above.

Issue triage

The following guidelines are used for triaging issues in content sites projects that are in-scope.

The triage guidelines use the product issue triage information as a basis. However, as the group structure and resources differ for the content sites, so do the guidelines.

A triage bot exists to help triage issues.

Types of issues and resourcing

Handbook issues typically fall under one of the following:

1. Content: Anything related to updating the text in the handbook, including required fixes.

1. Feature: New or an enhancement of how the handbook works, such as theme, or shortcode.

1. Operations: Theme, pipeline, local development, linters, other maintenance, and related documentation.

Issues may be considered a bug separately from the category when applicable (not a feature).

The team overseeing the content websites generally only resources operations issues, unless the issue is a blocker for contributing or using the handbook.

Priority

The priority label is used to indicate the importance of an issue and guide its scheduling.

Priority labels are expected to be set based on the needs of GitLab.

Priority can apply to any type of issue.

For bug issues, priority typically matches the severity.

Match the priority to the severity if uncertain.

Priority	Importance	Intention
Urgent	We will address this as soon as possible regardless of the limit on our team capacity. Our target resolution time is 30 days.	
High	We will address this soon and will provide capacity from our team for it. Our target resolution time is 30-90 days.	
Medium	We want to address this but don't have visibility when this will be addressed. No timeline designated.	
Low	We don't have visibility when this will be addressed. No timeline designated.	

We encourage contributions for all issues, especially priority 3 and 4 issues, in order to be addressed sooner.

Severity

Severity labels help us determine urgency and clearly communicate the impact of a ~"Handbook::Operations" issue on users.

The severity should be determined based the various factors in the table below. When an issue falls under multiple categories, use your best judgment.

Once you've determined a severity for an issue, add a note that explains in summary why you selected the severity you did. This will help future team members understand your rationale so they will know how to proceed with acting upon the issue.

~"Handbook::Operations" ~"type::bug"	~"hb-severity::1": Blocker	~"hb-severity::2": High	~"hb-severity::3": Medium	~"hb-severity::4": Low
General bugs	Broken feature with no workaround or any data-loss.	Broken feature with an unacceptably complex workaround.	Broken feature with a workaround.	Functionality is inconvenient.
Impact on users	Impacts 20% or more of users without an available workaround	Impacts 20% or more of users, but a reasonable workaround is available	AND/OR Impacts between 5%-20% of users without an available workaround	Impacts up to 20% of users with a reasonable workaround, or workaround not required.
Workaround is available or not needed.	Minimal impact on typical user's workflow.			
User experience problem	Users are blocked and/or likely to make risky errors.	Users are significantly delayed by the available workaround.	Users are self sufficient in completing the task with the workaround, but may be somewhat delayed.	Usability isn't ideal or there is a small cosmetic issue.

Timeline for resolution is based on priority.

Triage bot

Please see the relevant project readme for implementation information.

SECTION: about/style-guide.md

GitLab's general communications practices are detailed at GitLab Communication, but beyond those we do have some channel-specific guidance available.

See the list of resources below for links to other guides.

Handbook style guidance is covered primarily by the handbook markdown guide, and the editing handbook page.

In the absence of handbook-specific guidance, follow:

1. GitLab's Writing Style Guidelines, or
1. the Documentation Style Guide.

Related Resources

- GitLab Communication
- Markdown guide
- GitLab Documentation guidelines
 - Documentation style Guide
- GitLab style guides
- Pajamas Design System
- Marketing site content style guide
- Blog style guide

SECTION: about/editing-handbook/_index.md

This page focuses on how to edit the handbook.

Please see the handbook usage page for how and why we use the handbook, and other docs pages for further reference.

This page is intended to be complementary to Using GitLab at GitLab. Reviewing the GitLab 101 page may also be useful.

Have your own practical Handbook editing tips? Drop a video below!

> Tip Use AI to edit the handbook faster, and learn about Anthropic Claude and GitLab Duo.

Add yourself to the team page

For how to add yourself to the team page, see the add yourself to team guide.

Markdown formatting

For how to format markdown in the handbook, see the markdown style guide, which includes how to embed videos.

Use the Web IDE to edit the handbook

A typical workflow to edit the handbook:

1. Use the Web IDE to make changes.
1. Commit changes.
1. Create a merge request, assign reviewers.
1. Apply suggestions if any, or make further changes if needed.
1. "Set to auto-merge", or merge once all suggestions are resolved and you have the necessary approvals.

Web IDE steps to create a new MR

1. Navigate to the handbook page you want to edit. At the bottom, or on the right, click on the URL Edit this page to open the new Web IDE.
 - Alternatively, open the public handbook project and select Web IDE from the edit actions.This requires you to navigate into the source tree where the handbook files are located.

1. Familiarize yourself with the Web IDE:
 - The menu sections are located on the left
 - The explorer section provides the file tree for the project, which is opened automatically when editing a handbook file. (keyboard shortcut: Shift+Cmd+E)
 - The editing view is on the right side. On top, there are tabs to organize open files.
 - Additional meta information is provided at the bottom. Git branch, problems, current line number, spaces for indentation, encoding, end-of-line sequence, language mode, layout, notifications.
 - Tip: Hover over the menu sections and items to learn more about their purpose and keyboard shortcuts.

1. Edit the selected file, and try the Markdown preview. Cmd+Shift+P on macOS opens the Web IDE command palette to search for commands. For example, type Markdown, select Markdown: Open Preview to the Side and try the preview.
 - Note that the handbook markdown engine supports more rendering features than the Web IDE preview based on VS Code, and some items won't be rendered properly. Commit and create a draft merge request to view the handbook review apps to preview the page, such as to verify embedded images.

1. Upload new files by right-clicking and selecting Upload... for example images into static/images/handbook/ and the corresponding file tree, following the handbook directories.
 - Note the images section in the markdown guide.

1. Open the Source Control icon on the left menu which has a colored marker for counting the

file changes. Tip: Keyboard shortcut Ctrl+Shift+G.

1. Review the changed files, and once ready, specify a Git commit message. The message should be a brief (less than 120 characters) description of your changes.

1. Click on the · on the right side of the Commit and push button, and choose the Create new branch and commit option. Keyboard shortcut: Cmd+Enter.

1. Specify a new Git branch name (optional). Press Enter to continue. This action pushes the changes into the Git repository. The changes are persisted on the GitLab server and are immediately visible in the project view.

1. Check the pop-up at the bottom right, and click Create Merge Request to open the GitLab MR tab. Tip: The notification bell icon at the bottom brings back the notification, if accidentally dismissed.

1. Fill in the MR template, provide why the merge request is created, add labels (optionally using quick actions, /label ~handbook), assign yourself (/assign me), and set the branch to delete when merged. Optionally, use the draft option to prevent merging until ready.

Tip: Practice the keyboard shortcuts for your workflows: After making changes, Control+Shift+G to commit, Cmd+Enter, Cursor to select the branch, Enter, etc.

Note: The Cmd+w keyboard shortcut to close a file tab in the Web IDE gets overridden by the Chrome browser to close the browser tab. This is a known problem, use this shortcut carefully.

Web IDE steps to edit an existing MR

If you are editing an existing merge request (MR), there are 2 main differences from creating a new MR:

where to open the editor, and where to commit to.

1. Make additional changes to the MR: Select Code > Open in Web IDE on the upper right menu. This opens the Web IDE again to make changes. All changed files are opened automatically.

1. Make changes (see previous section for details), go to the Source Control area, add a commit message, and click on the Commit and push button. If you create a new branch, you will end up with a new MR instead of editing the existing one.

1. Optional: Follow the pop-up notification to navigate back to the MR. Alternatively, close the editor tab if you did not close the MR tab.

Video: Editing using the Web IDE and single file editor

Note: The video shows the old www.gitlab.com project. The editing process, however, is the same.

This video covers:

- Editing the marketing handbook as example
- Edit this page action using the Web IDE, overview walkthrough, editing the handbook, again creating a merge request with the same actions.
- Practical handbook edit tips shown in this handbook.

{{< youtube DJo4bQdawSo >}}

Historical videos

In these videos, we run through the GitLab Handbook with experts, uncovering how to best use the handbook in our day-to-day work, and learning best-practices for handbook editing along the way. They are meant to be helpful to understand generally how GitLab works, but the instructions cannot be followed exactly since the editor and repository in GitLab has changed.

1. Creating new handbook pages and multimedia embedding best-practices

1. How to move the location of a handbook page

1. Changing a page name and subsequent updates

1. Creating mermaid diagrams

1. Creating issue templates

1. Adding images to the handbook and handbook analytics

1. How to add a new directory and page to the handbook

1. How to rebase to fix a failing pipeline after fix is merged to main

1. Why a handbook pipeline is failing

1. How to identify why a pipeline is failing

Editing the handbook locally

1. Set up an SSH key to use with GitLab, if you have not previously done so.

1. Set up Git and clone the appropriate repository, such as the public handbook.

1. Use your editor of choice to make changes. If you use Visual Studio Code optionally along with the GitLab Workflow extension for VS Code, you can follow the Web IDE editing instructions.

1. Push the changes to GitLab.

1. Create a merge request.

Additional notes:

1. We don't need .gitkeep files in our handbook, they make it harder to quickly open a file in editors. Don't add them, and delete them when you see them.

If you want to install a local version of the website to test and preview things locally, see the handbook development page for instructions on how to set it up.

Preview changes on GitLab

To preview your changes, deploy the review app:

1. Wait for the pipeline on the merge request (MR) to complete.

1. Either on the "Pipelines" tab, or in the pipeline widget on the "Overview" tab of the MR, click on the circles that denote the different pipeline stages until you find the Stage:deploy and the pages job.

1. Next to the pages job name, click on the "play" button.

1. Once two pages jobs complete, in the pipeline widget on the "Overview" tab of the MR, click on the "View app" button.

1. You may need to navigate to your page by going to any page, then using the left navigation tree. Do not use the search for this.

- If you prefer to edit the URL, note that the review app uses a .html ending, so if your page is content/handbook/path/to/page.md, the URL will be review/app/public/handbook/path/to/path.html.

If you need the review app deployed after every change on a MR, you can add the ~"deploy-review-app-always" label to always trigger the job, including on MR creation.

For more information about how the review apps are configuration for the handbook projects, please see the Handbook Pages Deployment.

Naming pages and folder structure

The site uses the concept of page bundles, sections, and leaf pages. A section can have multiple leaf pages, which requires a index.md for the section. A page bundle is a single page with a group of images, which can be an index.md.

In general, Handbook URLs should describe their content and be as clean and easy to remember as possible.

Directories (folders) and pages should use lowercase a-z, hyphen -, and underscore . While Git and Hugo allow any UTF-8 character to be used in the file path, using other characters (such as a space) can cause issues with the pipeline, and thus, disallowed.

Section:

```
plain
section-dir/
|- index.md
|- leaf-page1.md
|- leaf-page2.md
```

Page bundle:

```
plain
page-name/
|- index.md
|- image1.png
|- image2.png
```

Section with a page bundle:

```
plain
section-dir/
|- index.md
|- leaf-page1.md
|- leaf-page2/
|  |- index.md
|  |- image.png
|- leaf-page3.md
```

Moving, deleting, or renaming a page

If you delete, rename, or move a page, you should:

1. update all existing links in the handbook and internal handbook.
 - Note: When searching for links, use the page name. Sometimes links use page-relative links, meaning it doesn't use the full path.
 - Please also consider searching for and updating handbook links in other places, such as docs and project templates in gitlab-com and gitlab-org.
1. add a redirect.

Editing page maintainers

On the right side of the page, there is a list "Maintainers".

The list is generated from the CODEOWNERS file in the relevant repository, such as in the handbook repository:

- Only specific users are listed. Members of a group are not listed.
- The list is generated based on the most specific path:
 - If users are specified for a directory and then for a specific page in that directory, the list only includes the users for the specific page.
 - If groups or subgroups are listed for a specific page without any specific users, the list of maintainers is empty.

Changes to the .gitlab/CODEOWNERS file require approval.

Review the bot comment for instructions on how to get the appropriate approval.

Editing content from shortcodes

When trying to edit content, you may encounter content that is "built" using shortcodes and data files.

Shortcodes look similar to `{{ % performance-indicators "uxdepartment" % }}`.

When you encounter content in a shortcode and want to edit it:

1. Look for the shortcode in the shortcodes folder.
 - In the above example, it's performance-indicators.
1. At the top of the shortcode file should be a comment on where to look for the relevant data file. Ensure you're using "Display source" to view the file.
 - For example, look in <https://gitlab.com/gitlab-com/www-gitlab-com/-/tree/master/data/performanceindicators> for the uxdepartment file.
 - If there is no comment, you can post in handbook to get help, and make a MR to add it.
1. Edit the file, being careful to match the existing formatting.

Team member merge requests being labeled as Community contributions

If you recently created a merge request that was labeled as a Community contribution, you can fix this mislabeling issue going forward by updating the GitLab username in your personal entry in the team member directory to match the GitLab account you use for work.

Use the team page editing instructions to find your team page entry file, and update the gitlab attribute (typically found on line 10) to be an exact match for the GitLab.com username you use for work.

Troubleshooting

404 errors with edit action

When clicking on Edit this page in the upper right corner on a handbook page in your browser, you might get a 404 error in the GitLab Web IDE.

As a team member, this problem can be related to an expired SAML session for your GitLab.com profile. In order to mitigate and solve the problem, click on View page source to trigger the SAML authentication with Okta again.

Alternatively, navigate into our GitLab.com profile into your To-Do list, or try to open a confidential issue, to trigger the authentication.

It can also be browser related: Try clearing the cache, open an incognito window (on macOS: cmd shift n), or use a different browser to test.

404 on new page

If a new page is created as part of a merge request, but the page is not showing up on the site, check the file name.

The most common issue is using index.md instead of index.md in a folder that has other pages. The other pages will not display.

See pages and folder structure for more information.

Images not loading properly

If you added new images and they are not loading properly in your review app, please review the Images section of the markdown guide.

Failing pipelines

To see why your pipeline is failing, there are two main places to look:

1. The latest comment by the bot on your merge request. It should have a list of all linter errors. However, build errors do not generate a comment.

1. Individual failed jobs. On the MR > "Pipelines" tab > select any red circle > select a failed job. Error messages are near